

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра вычислительной техники

ОТЧЁТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
ПО ДИСЦИПЛИНЕ «ПРОГРАММИРОВАНИЕ»

Факультет	ЗО	Преподаватель	Дубков Илья
Группа	ДТ-460а		Сергеевич
Студент	Пантюхин Артём		
	Евгеньевич		
Вариант	9		

Новосибирск, 2026 г.

Цель работы

Ознакомиться с шаблонными классами библиотеки STL. Изучить применение этих классов и их методов на практике.

Задание

Для встроенного типа (например, `int` или `char`) провести временной анализ заданных шаблонных классов на основных операциях: добавление, удаление, поиск, сортировка. Использовать итераторы для работы с контейнерами. Для получения времени выполнения операции засекать системное время начала и окончания операции и автоматически генерировать большое количество данных.

Вариант 9: Очередь; множество.

Выбран тип данных: `double`

Поскольку очередь не имеет поиска и сортировки, будет проведён временной анализ на операциях добавления и удаления

Поскольку множество автоматически сортируется при добавлении нового элемента, не предоставляя доступа к данному мутатору извне, будет проведён временной анализ на операциях добавления и удаления. Для операции поиска будет временной анализ будет состоять в нахождении среднего времени поиска среди всех присутствующих в множестве элементов и среднего времени поиска среди аналогичного размера набора отсутствующих в множестве элементов.

Исходный код

"main.cpp"

```
#include <cstdlib>
#include <ctime>
#include <iostream>
#include <queue>
#include <vector>
#include <chrono>
#include <set>

void print_title(const char* title) {
    std::cout << "\033[32m|\\" << title << " |\\|\033[0m" << std::endl;
}

void print_closing(const char* title) {
    std::cout << "\033[32m|^| " << title << " |^|\033[0m" << std::endl;
}

std::vector<double> get_values(std::size_t amount) {
    std::vector<double> values{};
    std::srand(std::time(0));
    values.resize(amount);
    for (std::size_t i = 0; i < amount; i++) {
        values[i] = (double)rand()*(double)std::rand()/(double)std::rand();
    }
    return values;
}

std::queue<double> test_queue_push(const std::vector<double>& values) {
    print_title("Queue push test");
    auto iter = values.cbegin();
    auto limit = values.cend();
    std::queue<double> test_container{};
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    for (; iter != limit; iter++) {
        test_container.push(*iter);
    }
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    auto value = std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    std::cout << "Time passed for "
        << float(values.size()) << " elements: "
        << value
        << "ns" << " == " << value/1.0e9 << "sec"
        << std::endl;
    print_closing("Queue push test done.");
    return test_container;
}

void test_queue_remove(std::queue<double> copy) {
    print_title("Queue remove test");
    std::size_t i = 0;
    std::size_t max = copy.size();
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    for (; i < max; i++) copy.pop();
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    auto value = std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    std::cout << "Time passed for "
```

```

        << float(max) << " elements: "
        << value
        << "ns" << " == " << value/1.0e9 << "sec"
        << std::endl;
    print_closing("Queue remove test done.");
}

void queue_test(std::size_t amount) {
    std::queue<double> source = test_queue_push(get_values(amount));
    test_queue_remove(source);
}

std::set<double> test_set_push(const std::vector<double>& values) {
    print_title("Set push test");

    auto iter = values.cbegin();
    auto limit = values.cend();
    std::set<double> test_container{};
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    for (; iter != limit; iter++) {
        test_container.insert(*iter);
    }
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    auto value = std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    std::cout << "Time passed for "
        << float(test_container.size()) << " elements: "
        << value
        << "ns" << " == " << value/1.0e9 << "sec"
        << std::endl;

    print_closing("Set push test done");
    return test_container;
}

void test_set_remove(std::set<double> copy) {
    print_title("Set remove test");
    std::size_t i = 0;
    std::size_t max = copy.size();
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    for (; i < max; i++) {
        copy.erase(copy.begin());
    }
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    auto value = std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    std::cout << "Time passed for "
        << float(max) << " elements: "
        << value
        << "ns" << " == " << value/1.0e9 << "sec"
        << std::endl;
    print_closing("Set remove test done");
}

void test_set_find(const std::set<double>& ref) {
    print_title("Set find test");
    double average = 0.0;
    for (auto iter = ref.begin(); iter != ref.end(); iter++) {
        std::chrono::steady_clock::time_point begin =
std::chrono::steady_clock::now();
        auto tmp = ref.find(*iter);

```

```

        std::chrono::steady_clock::time_point end =
std::chrono::steady_clock::now();
        average += std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    }
    average/=ref.size();
    std::cout << "Average search time for existing "
        << float(ref.size()) << " elements: "
        << average
        << "ns" << " == " << average/1.0e9 << "sec"
        << std::endl;

    average = 0.0;
    for (auto iter = ref.begin(); iter != ref.end(); iter++) {
        double nonexistent_value = (double)rand()/(double)rand();
        while (ref.count(nonexistent_value) != 0) {
            nonexistent_value = (double)rand()/(double)rand();
        }
        std::chrono::steady_clock::time_point begin =
std::chrono::steady_clock::now();
        auto tmp = ref.find(nonexistent_value);
        std::chrono::steady_clock::time_point end =
std::chrono::steady_clock::now();
        average += std::chrono::duration_cast<std::chrono::nanoseconds>(end-
begin).count();
    }
    average/=ref.size();
    std::cout << "Average search time for nonexistent "
        << float(ref.size()) << " elements: "
        << average
        << "ns" << " == " << average/1.0e9 << "sec"
        << std::endl;
    print_closing("Set find test done");
}

void set_test(std::size_t amount) {
    std::set<double> source = test_set_push(get_values(amount));
    test_set_remove(source);
    test_set_find(source);
}

void queue_test_up_to(std::size_t max_amount) {
    for (std::size_t amount = 1; float(amount) < max_amount; amount*=10) {
        queue_test(amount);
    }
}

void set_test_up_to(std::size_t max_amount) {
    for (std::size_t amount = 1; float(amount) < max_amount; amount*=10) {
        set_test(amount);
    }
}

void test_up_to(std::size_t max_amount) {
    queue_test_up_to(max_amount);
    std::cout << std::endl << std::endl;
    set_test_up_to(max_amount);
}

int main(void) {
    test_up_to(1e7);
    return 0;
}

```

Выводы

Была реализована программа, выполняющая временной анализ на доступных операциях контейнеров библиотеки STL очередь и множество. По результатам тестирования обнаружено, что, по крайней мере, на наборах случайных элементов размером менее чем $1 \cdot 10^9$ время выполнения возрастает практически линейно. Время поиска элемента в случае с множеством возрастает согласно ожидаемому от двоичного дерева поиска, соответствуя временной сложности $O(\log(n))$, где n – количество элементов.

В результате выполнения данной лабораторной работы была изучена логика работы и функционал стандартной библиотеки шаблонов STL, получено представление об эффективности реализаций классов очереди и множества из данной библиотеки.